

```
obj-m += VTK_module_tutorial_1.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Save this in the same directory in which you have saved your C program. Now you can *cd* to that directory from the terminal and issue the *make* command:

```
aasisvinayak@GNU-BOX:~/Desktop/modules$ make
make -C /lib/modules/2.6.31-17-generic-pae/build M=/home/aasisvinayak/Desktop/modules modules
make[1]: Entering directory `/usr/src/linux-headers-2.6.31-17-generic-pae'
CC [M] /home/aasisvinayak/Desktop/modules/VTK_module_tutorial_1.o
Building modules, stage 2.
MODPOST 1 modules
CC /home/aasisvinayak/Desktop/modules/VTK_module_tutorial_1.mod.o
LD [M] /home/aasisvinayak/Desktop/modules/VTK_module_tutorial_1.ko
make[1]: Leaving directory `/usr/src/linux-headers-2.6.31-17-generic-pae'
```

You can now see a few other files created in the directory (refer to Figure 1).

If you are a Java developer, then you must have used *getters* and *setters* while meddling with Java beans. Here, in one of the automatically generated files (source), you can find something similar:

```
#include <linux/module.h>
#include <linux/vermagic.h>
#include <linux/compiler.h>

MODULE_INFO(vermagic, VERMAGIC_STRING);

struct module __this_module
__attribute__((section("gnu.linkonce.this_module"))) = {
    .name = KBUILD_MODNAME,
    .init = init_module,
#ifdef CONFIG_MODULE_UNLOAD
    .exit = cleanup_module,
#endif
    .arch = MODULE_ARCH_INIT,
};

static const struct modversion_info ____versions[]
__used
__attribute__((section("__versions"))) = {
    { 0x4e0ead741, "module_layout" },
    { 0xb224f8e2, "param_get_short" },
    { 0x4333eadb, "param_set_short" },
    { 0x6980fe91, "param_get_int" },
    { 0xdf964b25, "param_set_int" },
    { 0x8bd5b603, "param_get_long" },
    { 0x3457cb68, "param_set_long" },
    { 0x1344088, "param_get_charp" },
    { 0x8ad0684, "param_set_charp" },
    { 0x43ab66c3, "param_array_get" },

```



Figure 1: More files are created after running make

```
{ 0x45947727, "param_array_set" },
{ 0xb72397d5, "printk" },
};

static const char __module_depends[]
__used
__attribute__((section("__modinfo"))) =
"depends=";

MODULE_INFO(srcversion, "FEC7A3AF90AAP6E9B3EA05B");
```

Though we have seen a few aspects/properties of these files already, we will revisit these areas and cover them in depth. Since this is an introductory column for the new segment, I'm avoiding the minute details.

Now you can load the module to the kernel and check if it is getting loaded properly:

```
aasisvinayak@GNU-BOX:~/Desktop/modules$ sudo insmod VTK_module_tutorial_1.ko
aasisvinayak@GNU-BOX:~/Desktop/modules$ lsmod
Module      Size Used by
VTK_module_tutorial_1 2916 0
msdos       7836 0
fat         51452 1 msdos
<<<OUTPUT TRUNCATED>>>
```

Now let's test it by passing a parameter. We will pass a valid argument at first and then we will pass an invalid one.

```
aasisvinayak@GNU-BOX:~/Desktop/modules$ sudo insmod VTK_module_tutorial_1.ko string_entry="hooray" integerArray=1
aasisvinayak@GNU-BOX:~/Desktop/modules$ sudo rmmod VTK_module_tutorial_1.ko string_entry="hoo" integerArray=1 invalid_entry="null"
insmod: error inserting 'VTK_module_tutorial_1.ko': -1 Unknown symbol in module
```

We have not yet covered some other basic elements like checking the modules (loaded) regularly and removing (cleaning) the ones that are not in use. We will cover these topics in the next segment.

Happy kernel hacking! 

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com